LAPORAN PRAKTIKUM
ANALISIS KOMPLEKSITAS ALGORITMA

Disusun Oleh:

MUHAMMAD FATHUR ARRIZKI

NIM **2025573010052**

KELAS **: TI-1D**

JURUSAN **: Teknologi Informasi dan Komputer**

PRODI **: Teknik Informatika**

DOSEN PENGAJAR **: M. Reza Zulman, SST. M.Sc**

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE

2026

KATA PENGANTAR

Puji dan syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum yang berjudul “Analisis Kompleksitas Algoritma Big O Notation, Time & Space Complexity” dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum serta sebagai sarana untuk memahami konsep dasar analisis algoritma, khususnya mengenai Big O Notation, kompleksitas waktu (*time complexity*), dan kompleksitas ruang (*space complexity*). Melalui praktikum ini, saya dapat mempelajari bagaimana efisiensi suatu algoritma diukur serta memahami perbandingan performa antara algoritma yang berbeda.

saya menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca mengenai pentingnya analisis kompleksitas dalam pengembangan algoritma.

Buketrata, 23, April, 2026

Muhammad Fathur Arrizki

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULIUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Tujuan Praktikum 2
BAB II LANDASAN TEORI 3
2.1 Algoritma 3
2.2 Analisis Kompleksitas Algoritma 3
2.3 Big O Notation 3
2.4 Time Complexity 4
2.5 Space Complexity (Kompleksitas Ruang) 4
2.6 Struktur Data Set 5
2.7 Perbandingan Algoritma 5
2.8 Trade-Off Time dan Space 5
BAB III LANGKAH- LANGKAH PRAKTIKUM 6
3.1 Alat dan Bahan 6
3.2 Langkah- Langkah 6
3.3 Summit ke Github 10
BAB IV PENUTUPAN 12
4.1 Kesimpulan 12
4.2 Saran 12

BAB I
PENDAHULIUAN

1.1 Latar Belakang

Dalam era digital saat ini, pengolahan data menjadi salah satu aspek penting dalam pengembangan perangkat lunak. Seiring dengan meningkatnya data yang harus diproses, efisiensi algoritma menjadi faktor krusial yang menentukan kinerja suatu sistem. Algoritma yang tidak efisien dapat menyebabkan waktu eksekusi yang lama serta penggunaan sumber daya yang tidak optimal, sehingga berdampak pada performa aplikasi secara keseluruhan.

Untuk mengatasi hal tersebut, diperlukan suatu metode analisis yang mampu mengevaluasi kinerja algoritma secara objektif dan sistematis. Big O Notation merupakan pendekatan yang umum digunakan untuk mengukur tingkat kompleksitas algoritma berdasarkan pertumbuhan ukuran input. Melalui analisis ini, dapat diketahui bagaimana perubahan jumlah data mempengaruhi waktu eksekusi (*time complexity*) dan penggunaan memori (*space complexity*).

Pemahaman terhadap konsep kompleksitas algoritma sangat penting, khususnya dalam menentukan solusi yang paling efisien dari berbagai alternatif yang tersedia. Dalam praktiknya, seringkali terdapat lebih dari satu algoritma untuk menyelesaikan suatu permasalahan, namun dengan tingkat efisiensi yang berbeda. Oleh karena itu, analisis perbandingan antar algoritma menjadi langkah penting dalam proses pengambilan keputusan.

Pada praktikum ini, dilakukan implementasi dan analisis terhadap beberapa algoritma dengan kompleksitas yang berbeda, termasuk perbandingan antara metode pencarian pasangan angka dalam array menggunakan pendekatan sederhana dan pendekatan yang

lebih optimal. Melalui kegiatan ini, diharapkan mahasiswa dapat memahami konsep Big O Notation secara lebih mendalam serta mampu menerapkannya dalam pengembangan program yang efisien.

1.2 Rumusan Masalah

1. Apa itu Big O Notation dalam analisis algoritma?
2. Bagaimana cara menentukan **time complexity** dan **space complexity** pada algoritma?
3. Bagaimana perbedaan tingkat efisiensi antara algoritma dengan kompleksitas yang berbeda?
4. Bagaimana implementasi algoritma dalam mencari pasangan angka dalam array yang jumlahnya sama dengan nilai target?
5. Algoritma manakah yang lebih efisien antara metode sederhana (nested loop) dan metode menggunakan struktur data set?

1.3 Tujuan Praktikum

1. Memahami konsep Big O Notation dalam analisis kompleksitas algoritma.
2. Mengetahui cara menentukan *time complexity* dan *space complexity* pada suatu algoritma.
3. Menganalisis perbedaan tingkat efisiensi antara beberapa algoritma dengan kompleksitas yang berbeda.
4. Mengimplementasikan algoritma untuk mencari pasangan angka dalam array yang jumlahnya sesuai dengan nilai target.
5. Membandingkan kinerja algoritma menggunakan pendekatan nested loop dan pendekatan menggunakan struktur data *Set*.

BAB II
LANDASAN TEORI

2.1 Algoritma

Algoritma adalah serangkaian langkah-langkah logis dan sistematis yang digunakan untuk menyelesaikan suatu permasalahan. Dalam dunia pemrograman, algoritma menjadi dasar dalam pembuatan program agar dapat berjalan dengan baik dan menghasilkan output yang diinginkan. Sebuah algoritma yang baik tidak hanya benar secara logika, tetapi juga efisien dalam penggunaan waktu dan sumber daya**.**

2.2 Analisis Kompleksitas Algoritma

Analisis kompleksitas algoritma merupakan proses untuk mengevaluasi efisiensi suatu algoritma berdasarkan kebutuhan waktu dan ruang (memori) terhadap ukuran input. Analisis ini penting untuk mengetahui performa algoritma ketika dijalankan pada data dalam jumlah besar.

Kompleksitas algoritma umumnya dibagi menjadi dua, yaitu time complexity dan space compBi

2.3 Big O Notation

Big O Notation adalah notasi yang digunakan untuk menggambarkan batas atas (*upper bound*) dari pertumbuhan kompleksitas suatu algoritma. Notasi ini menunjukkan bagaimana kinerja algoritma berubah seiring dengan bertambahnya ukuran input (n).

Big O tidak memperhatikan konstanta dan faktor kecil, melainkan fokus pada pertumbuhan terbesar dari suatu fungsi.

Contoh notasi Big O:

- **O(1)** → Konstan
  Waktu eksekusi tidak berubah walaupun data bertambah

- **O(n)** → Linear
  Waktu bertambah sedikit saja walaupun data sangat besar

- **$O(n^2)$** → Kuadrat

Waktu bertambah sebanding dengan jumlah data

$O(2^n)$ – Eksponensial
Waktu meningkat sangat drastic

O(n!) – Faktorial
Kompleksitas paling buruk

- **O(log n)** → Logaritmik
  jumlah langkah bertambah sangat lambat meskipun data bertambah besar.

  $O(n^3)$ → Kubik
  jumlah operasi bertambah sangat cepat (pangkat tiga) terhadap jumlah data.

2.4 Time Complexity

Time complexity adalah ukuran yang digunakan untuk mengetahui berapa lama waktu yang dibutuhkan oleh suatu algoritma untuk menyelesaikan masalah berdasarkan ukuran input.

Semakin kecil nilai kompleksitas, maka semakin cepat algoritma tersebut.

Contoh:

- **O(1)** → Waktu tetap, tidak dipengaruhi input
- **O(n)** → Waktu bertambah sebanding dengan input
- **O(n^2)** → Waktu bertambah sangat cepat seiring input

2.5 Space Complexity (Kompleksitas Ruang)

Space complexity adalah ukuran yang digunakan untuk mengetahui jumlah memori yang dibutuhkan oleh suatu algoritma selama proses eksekusi.

Penggunaan memori ini meliputi variabel, struktur data, dan ruang tambahan lainnya yang digunakan oleh program.

Contoh:

- **O(1)** → Menggunakan memori tetap
- **O(n)** → Menggunakan memori sesuai jumlah input.

2.6 Struktur Data Set

Set adalah salah satu struktur data yang digunakan untuk menyimpan kumpulan nilai unik (tidak ada duplikasi). Dalam bahasa pemrograman JavaScript, *Set* memungkinkan pencarian data dilakukan dengan cepat.

Operasi utama pada *Set* seperti penambahan dan pengecekan keberadaan elemen memiliki kompleksitas waktu rata-rata O(1), sehingga sangat efisien untuk digunakan dalam pencarian data.

2.7 Perbandingan Algoritma

Dalam menyelesaikan suatu permasalahan, seringkali terdapat lebih dari satu algoritma yang dapat digunakan. Setiap algoritma memiliki kelebihan dan kekurangan masing-masing.

Sebagai contoh, dalam pencarian pasangan angka dalam array:

- Metode nested loop memiliki kompleksitas waktu $O(n^2)$ namun tidak membutuhkan memori tambahan.
- Metode menggunakan Set memiliki kompleksitas waktu O(n) namun membutuhkan memori tambahan sebesar O(n).

Hal ini menunjukkan adanya *trade-off* antara efisiensi waktu dan penggunaan ruang.

2.8 Trade-Off Time dan Space

Trade-off adalah kondisi di mana peningkatan pada satu aspek menyebabkan penurunan pada aspek lainnya. Dalam konteks algoritma, trade-off sering terjadi antara waktu eksekusi dan penggunaan memori. Algoritma yang lebih cepat biasanya membutuhkan memori tambahan,

BAB III
LANGKAH- LANGKAH PRAKTIKUM

3.1 Alat dan Bahan

1. Visual Studio Code
2. Laptop
3. Jaringan wifi

3.2 Langkah- Langkah

1. Cd /d
2. Cd “/d/nama folder utama
3. Cd nama_repository
4. Mkdir praktikum-5
5. Cd praktikum-5
6. Npm init -y
7. Selanjutnya buat file : touch 01-intro-kompleksitas.js
8. Ketik programnya dan lihat outputnya

```
===Perbandingan waktu Eksekusi===
0(1) jumlahkanRumus : 0 ms
0(n) jumlahkanLinear : 1 ms
0 (n^2 cariPasangan : 0 ms

hasil sama? true
PS D:\Praktikum semester 2\2025573010049_strukturdataalgoritma\praktikum-5>
```

9. Kemudian buat file : touch 02-descructuring-spread.js
10. Ketik programnya dan lihat outpunya :

```
PS D:\Praktikum semester 2\2025573010049_strukturdataalgoritma\praktikum-5> node "d:\Praktikum semester 2\2025573010049_s
a\praktikum-5\02-descructuring-spread.js"
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 999

=== O(n2) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
108 ms
Waktu fib(35) O(n) memoization:
0 ms
PS D:\Praktikum semester 2\2025573010049_strukturdataalgoritma\praktikum-5>
```

11. Ketik touch 03-space-complexity.js
12. Buat program di file tersebut dan lihat outputnya

```
PS D:\Praktikum semester 2\2025573010049_strukturdataalgoritma\praktikum-5> node "d:\Praktikum semester 2\2025573010049_st
a\praktikum-5\tempCodeRunnerFile.js"
Hasil Lambat: [ 2, 7 ]
Hasil Cepat : [ 2, 7 ]
```

13. Ketik 04-refactor
14. Ketik program dan lihat outpunya

```
C:\Program Files\nodejs\node.exe .\04-refactor.js
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: {a: 3, b: 2, c: 1, d: 1}
```

15. Kerjakan latihan-1, tapi karna yang dibutuhkan 1000, maka output yang berjalan membutuhkan waktu sangat lama.
16. Kerjakan tugas 1 dan tugas 2.
17. Output tugas 1

```
[ 2, 3 ]
[ 2, 3 ]
[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
true
true
intersectionLambat: 16 ms
intersectionCepat: 0 ms
tripletLambat: 14 ms
tripletCepat: 1 ms
PS D:\Praktikum semester 2\2025573010049_strukturdataalgoritma\praktikum-5>
```

18. Output tugas 2.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Praktikum semester 2\2025573010049_strukturdataalgoritma\praktikum-5> node
a\praktikum-5\tugas\tugas-2.js"

Ukuran Data: 100

===konstan===
O(1): 0.031ms

===Linear===
O(n): 0.051ms

===Linear mithm===
O(n log n): 0.304ms

===kuadratik
O(n2): 0.195ms

Ukuran Data: 500

===konstan===
O(1): 0.019ms

===Linear===
O(n): 0.033ms

===Linear mithm===
O(n log n): 0.617ms

===kuadratik
O(n2): 0.831ms
```

```
Ukuran Data: 1000

===konstan===
O(1): 0.015ms

===Linear===
O(n): 0.06ms

===Linear mithm===
O(n log n): 0.462ms

===kuadratik
O(n2): 0.624ms

Ukuran Data: 5000

===konstan===
O(1): 0.006ms

===Linear===
O(n): 0.011ms

===Linear mithm===
O(n log n): 22.495ms

===kuadratik
O(n2): 27.557ms

Ukuran Data: 10000

===konstan===
O(1): 0.008ms
```

```
===Linear===
O(n): 0.02ms

===Linear mithm===
O(n log n): 70.166ms

===kuadratik
O(n2): 101.61ms
PS D:\Praktikum semester 2\2025573010049_strukturdataalgoritma\praktikum-5> no
a\praktikum-5\tugas\tugas-2.js"

Ukuran Data: 100

===konstan===
O(1): 0.045ms

===Linear===
O(n): 0.057ms

===Linear mithm===
O(n log n): 0.239ms

===kuadratik===
O(n2): 0.236ms

Ukuran Data: 500
```

```
===Linear mithm===
O(n log n): 1.098ms

===kuadratik===
O(n2): 0.876ms

Ukuran Data: 1000

===konstan===
O(1): 0.596ms

===Linear===
O(n): 0.103ms

===Linear mithm===
O(n log n): 0.747ms

===kuadratik===
O(n2): 1.012ms

Ukuran Data: 5000

===konstan===
O(1): 0.008ms
```

```
===kuadratik===
O(n2): 25.585ms

Ukuran Data: 10000

===konstan===
O(1): 0.01ms

===Linear===
O(n): 0.023ms

===Linear mithm===
O(n log n): 66.385ms

===kuadratik===
O(n2): 102.671ms
PS D:\Praktikum semester 2\2025573010049_strukturdataalgoritma\praktikum-5> node "d:\P
a\praktikum-5\tugas\tugas-2.js"

Ukuran Data: 100

===konstan===
O(1): 0.035ms

===Linear===
O(n): 0.079ms

===Linear mithm===
O(n log n): 0.317ms

===kuadratik===
O(n2): 0.188ms
```

```
Ukuran Data: 500

===konstan===
O(1): 0.029ms

===Linear===
O(n): 0.036ms

===Linear mithm===
O(n log n): 0.512ms

===kuadratik===
O(n2): 0.499ms

Ukuran Data: 1000

===konstan===
O(1): 0.043ms

===Linear===
O(n): 0.069ms

===Linear mithm===
O(n log n): 0.443ms

===kuadratik===
O(n2): 0.581ms

Ukuran Data: 5000

===konstan===
O(1): 0.006ms
```

```
===kuadratik===
O(n2): 0.588ms

Ukuran Data: 5000

===konstan===
O(1): 0.006ms

===Linear===
O(n): 0.008ms

===Linear mithm===
O(n log n): 17.425ms

===kuadratik===
O(n2): 25.527ms

Ukuran Data: 10000

===konstan===
O(1): 0.006ms

===Linear===
O(n): 0.119ms

===Linear mithm===
O(n log n): 66.044ms

===kuadratik===
O(n2): 103.167ms
```

```
===Linear mithm===
O(n log n): 0.222ms

===kuadratik===
O(n2): 0.313ms

Ukuran Data: 500

===konstan===
O(1): 0.02ms

===Linear===
O(n): 0.029ms

===Linear mithm===
O(n log n): 0.529ms

===kuadratik===
O(n2): 0.874ms

Ukuran Data: 1000

===konstan===
O(1): 0.018ms

===Linear===
O(n): 0.063ms

===Linear mithm===
O(n log n): 1.191ms
```

```
===Linear===
O(n): 0.01ms

===Linear mithm===
O(n log n): 18.693ms

===kuadratik===
O(n2): 28.957ms

Ukuran Data: 10000

===konstan===
O(1): 0.008ms

===Linear===
O(n): 0.015ms

===Linear mithm===
O(n log n): 67.163ms

===kuadratik===
O(n2): 117.379ms
PS D:\Praktikum semest> node "d:\Praktikum semester 2\2025573010049_strukturd

Ukuran Data: 100

===konstan===
O(1): 0.036ms

===Linear===
O(n): 0.046ms
```

3.3 Summit ke Github

1. Setelah kita membuat file tugas di gitbush, dan sudah mengerjakanya, maka selanjutnya summit ke github.
2. Ketik cd ..
3. Git status
4. Git add praktikum-5/

```
LENOVO@LAPTOP-S39ET2DT MINGW64 ~ (main)
$ cd /d

LENOVO@LAPTOP-S39ET2DT MINGW64 /d
$ cd "/d/praktikum semester 2"

LENOVO@LAPTOP-S39ET2DT MINGW64 /d/praktikum semester 2
$ cd 2025573010049_strukturdataalgoritma

LENOVO@LAPTOP-S39ET2DT MINGW64 /d/praktikum semester 2/2025573010049_strukturdat
aalgoritma (main)
$ cd praktikum-5

LENOVO@LAPTOP-S39ET2DT MINGW64 /d/praktikum semester 2/2025573010049_strukturdat
aalgoritma/praktikum-5 (main)
$ cd..
bash: cd..: command not found

LENOVO@LAPTOP-S39ET2DT MINGW64 /d/praktikum semester 2/2025573010049_strukturdat
aalgoritma/praktikum-5 (main)
$ cd ..

LENOVO@LAPTOP-S39ET2DT MINGW64 /d/praktikum semester 2/2025573010049_strukturdat
aalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   praktikum-2/04-kondisional.js
        modified:   praktikum-2/tugas/tugas-1.js
        modified:   praktikum-2/tugas/tugas-2.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        praktikum-5/

no changes added to commit (use "git add" and/or "git commit -a")

LENOVO@LAPTOP-S39ET2DT MINGW64 /d/praktikum semester 2/2025573010049_strukturdat
aalgoritma (main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced
by CRLF the next time Git touches it
```

5. Git commit -m “Praktikum 5: Analisis Kompleksitas Algoritma”

```
LENOVO@LAPTOP-S39ET2DT MINGW64 /d/praktikum semester 2/2025573010049_strukturdat
aalgoritma (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main 5488ee0] Praktikum 5: Analisis Kompleksitas Algoritma
 9 files changed, 422 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-descructuring-spread.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tempCodeRunnerFile.js
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js

LENOVO@LAPTOP-S39ET2DT MINGW64 /d/praktikum semester 2/2025573010049_strukturdat
aalgoritma (main)
$ git push origin main
```

6. Git push origin main

```
LENOVO@LAPTOP-S39ET2DT MINGW64 /d/praktikum semester 2/2025573010049_strukturdat
aalgoritma (main)
$ git push origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 8 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (13/13), 5.17 KiB | 1.29 MiB/s, done.
Total 13 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/MifthaArtika4/2025573010049_strukturdataalgoritma.git
   5479e56..5488ee0  main -> main
```

7. Dan periksa ke akun github kamu, apakah sudah ada?

BAB IV
PENUTUPAN

4.1 Kesimpulan

Berdasarkan pembahasan tentang *Big O Notation*, *Time Complexity*, dan *Space Complexity*, dapat disimpulkan bahwa analisis kompleksitas algoritma sangat penting untuk menilai efisiensi suatu program. Big O Notation digunakan untuk menggambarkan pertumbuhan kebutuhan waktu dan memori terhadap ukuran input, sehingga memudahkan perbandingan antar algoritma. Time Complexity menunjukkan seberapa cepat algoritma berjalan, sedangkan Space Complexity menunjukkan seberapa besar memori yang dibutuhkan. Algoritma yang baik adalah yang mampu menyelesaikan masalah dengan waktu eksekusi yang cepat dan penggunaan memori yang efisien, terutama saat menangani data dalam jumlah besar.

4.2 Saran

1. Lebih sering berlatih menganalisis berbagai jenis algoritma (linear, quadratic, logarithmic, dll).
2. Mengimplementasikan algoritma dalam kode program agar memahami pengaruhnya terhadap waktu eksekusi secara nyata.
3. Menggunakan tools atau fitur benchmarking untuk mengukur performa algoritma.
4. Memilih algoritma yang sesuai dengan kebutuhan, tidak hanya berdasarkan kecepatan tetapi juga efisiensi memori.
5. Terus mempelajari teknik optimasi algoritma agar dapat membuat program yang lebih efektif dan scalable